

# Entrenar redes neuronales cuánticas para hardware limitado por T1: qué hace y qué no hace el filtro de simetría qg

Vicente Humberto Monteverde  
Universidad del Museo Social Argentino (UMSA), Argentina  
ORCID 0000-0001-8884-4811

Borrador del 3 de octubre de 2026

## Resumen

Una red neuronal cuántica (QNN) construida con compuertas que conservan el peso de Hamming mantiene sus datos en un sector de peso, así que los disparos que salieron del sector pueden descartarse en la lectura. Este es el filtro qg de la librería qang [1, 2]. Mostramos que, con igual amortiguamiento de amplitud (T1) en todos los qubits, la lectura filtrada de una red así es exactamente la lectura sin ruido, en cualquier sector de peso. La fracción de disparos conservados es  $(1 - \gamma)^{kd}$  para peso  $k$  y profundidad  $d$ , y entrenar bajo T1 con el filtro da exactamente los parámetros del entrenamiento sin ruido. Luego medimos cuánto vale esto en cuatro conjuntos de clasificación pequeños. Cada resultado se reporta con el filtro y sin él, y cada predicción quedó registrada antes de su corrida, fallas incluidas. En 60 corridas por modelo, una red entrenada en simulador y ejecutada bajo T1 gana +2.5 puntos con el filtro en peso 1 (IC 95 % [0.8, 4.1]) y +16.5 puntos en peso 2 ([11.8, 21.2]), y recupera exactamente su precisión sin ruido. Una red entrenada con el ruido calibrado sin el filtro la alcanza (diferencias de 0.1 puntos), y el filtro no corrige ni el desfase ni un T1 distinto entre qubits, aunque solo pierde menos de un punto hasta una dispersión de  $\pm 80\%$  en  $1/T_1$ . Con entrenamiento con ruido y una lectura más rica, el modelo sin filtro es 1.2 puntos mejor: descartar disparos decaídos puede costar precisión. El déficit del peso 2 viene de la codificación por productos de pares, no de la lectura. Con los modelos de ruido de equipos de IBM e IonQ, el filtro reduce el error del  $qgz$  medido entre 2.9 y 3.6 veces. Estos modelos son simulables clásicamente y no se afirma ventaja cuántica. El filtro es una manera de hacer válido el entrenamiento sin ruido bajo relajación, con un costo en disparos conocido de antemano. Los modelos se publican como `qang.qml` (versión 0.6.0, `pip install qang`).

## 1 Introducción

Los clasificadores cuánticos variacionales se entrenan en simuladores y se ejecutan en equipos con ruido, o se entrenan directamente con el ruido del equipo. El entrenamiento consciente del ruido [3] es el remedio habitual cuando ambos difieren. La verificación de simetría [4, 5] es otro remedio: cuando el circuito ideal conserva una cantidad, se descartan los resultados que la violan. El proyecto qang expresa las lecturas de qubits en unidades qg ( $qgz = \langle \sigma_z \rangle$  y sus compañeras) [1]. Construyó un testigo de media de registro y un filtro por disparo para circuitos que conservan un peso de Hamming [2]. Esta nota hace una pregunta acotada: para una QNN que conserva el peso, ¿cuánto cambia ese filtro la precisión de un clasificador en hardware limitado por T1, y cuándo deja de importar?

Respondemos con simulación exacta, un protocolo fijo y una comparación explícita. Cada lectura se toma con qang (filtrada) y sin qang (cruda), sobre los mismos parámetros entrenados o la misma inicialización. Las predicciones de cada estudio quedaron registradas en el repositorio antes de la corrida que las puso a prueba. De 44 predicciones pre-registradas en los estudios que se resumen aquí, 33 se cumplieron y 11 fallaron; las fallas se reportan junto a los aciertos.

## 2 Modelo y teoría

**QNN que conserva el peso.** En  $n$  qubits, una compuerta RBS (*reconfigurable beam splitter*) actúa sobre un par  $(a, b)$  como una rotación de Givens entre  $|10\rangle$  y  $|01\rangle$  y deja fijos  $|00\rangle$  y  $|11\rangle$ , así que conserva el peso de Hamming [6]. Usamos  $n = 5$  y tres capas de compuertas RBS sobre los pares  $\{(0, 1), (2, 3)\}$ ,  $\{(1, 2), (3, 4)\}$ ,  $\{(4, 0)\}$ . Eso da 15 ángulos y profundidad  $d = 9$  (subcapas). La lectura es  $z = \sum_i c_i qg_Z^{(i)} + b$  con  $c_i, b$  entrenables y pérdida logística.

**Codificaciones.** Con cuatro variables  $x \in [-1, 1]^4$  sea  $v = (x, 1)/\|(x, 1)\|$ ; la componente constante conserva  $\|x\|$ . El modelo E (peso 1) pone  $v_i$  en el estado base con el qubit  $i$  excitado. El modelo W (peso 2) pone  $v_i v_j$  en el estado con los qubits  $i < j$  excitados, normalizado. Los sectores tienen dimensión 5 y 10.

**El filtro.** Con qang, la lectura conserva solo los disparos cuyo peso es igual al peso de entrada  $k$  y renormaliza (`qang.sectors.filter_distribution`). Sin qang, se usan todos los disparos.

**F1/F4: exactitud con T1 igual.** Sea un amortiguamiento de amplitud con probabilidad  $\gamma$  en cada qubit después de cada subcapa. Su operador de Kraus sin salto multiplica un estado base con  $k$  excitaciones por  $(1 - \gamma)^{k/2}$ : el mismo factor para todos los estados del sector de peso  $k$ . Todo salto baja el peso y sale del sector. Las compuertas que conservan el peso nunca devuelven un estado rebajado. Por eso, condicionado a que el peso sea  $k$ , el estado es exactamente el sin ruido, y

$$\text{fracción conservada} = (1 - \gamma)^{kd}. \quad (1)$$

Para  $\gamma = 0.08$  y  $d = 9$  es 0.472 en peso 1 y 0.223 en peso 2. Si el amortiguamiento cambia entre qubits, el factor sin salto es  $\prod_{q \in S} (1 - \gamma_q)^{1/2}$  sobre el conjunto excitado  $S$ , que no es uniforme, y se pierde la exactitud. El desfase conserva el peso, así que el filtro no lo ve.

**F3: entrenamiento.** Como la lectura filtrada bajo T1 es igual a la sin ruido para cualquier valor de los parámetros, la pérdida y todos los gradientes son idénticos. Entrenar bajo T1 con el filtro da los parámetros del entrenamiento sin ruido, con la misma inicialización.

**F2: simulabilidad.** En peso 1 la red es una matriz ortogonal  $O$  de  $5 \times 5$  sobre  $v$ , y  $z$  es una forma cuadrática en  $v$ . En peso  $k$  el sector tiene dimensión  $\binom{n}{k}$ , polinomial para  $k$  fijo. Estos modelos se pueden simular clásicamente. Si versiones más grandes y de mayor peso dejarían de serlo es otra pregunta [7] que esta nota no aborda.

## 3 Métodos

Conjuntos de datos (binarios): iris (versicolor contra virginica), cáncer de mama, vino (clases 0 contra 1) y dígitos (3 contra 8), con 100 muestras por clase cuando hay. Cada corrida usa una partición estratificada 70/30, estandarización, PCA a cuatro componentes donde hace falta y escalado a  $[-1, 1]$ ; una muestra de prueba vale 2–3 puntos. El entrenamiento usa Adam durante 120 épocas, con gradientes por diferencias centrales en los ángulos. Las condiciones de ruido se aplican después de cada subcapa: T1 igual con  $\gamma = 0.08$ ; T1 desigual con  $\gamma_q$  de 0.04 a 0.12 (media 0.08); y T1 igual más un cambio de fase con probabilidad 0.03. La simulación es exacta. T1 y desfase mantienen la matriz densidad diagonal por bloques según el peso de Hamming, así que se propaga un bloque por peso; se verificó contra la matriz densidad completa a  $10^{-12}$ . Protocolo: el script de cada estudio, con sus predicciones escritas en el encabezado, se registró antes de usar la semilla de su corrida reportada; el código se depuró con otra semilla y pocas épocas. Los hashes de los commits están en las notas de investigación [1].

## 4 Resultados

**Estudios con una semilla.** La QNN que conserva el peso y la norma llega a una precisión media de 0.960 sobre los cuatro conjuntos. Queda a la par de una QNN estándar de cuatro qubits que no conserva el peso (0.952), de una máquina de vectores soporte RBF (0.957) y de una red MLP pequeña (0.961). Sin la componente de norma, la misma red obtiene 0.791. Entrenada sin ruido y ejecutada bajo T1 con el filtro, conserva exactamente su precisión sin ruido. Entrenar bajo T1 con el filtro reproduce el entrenamiento sin ruido a  $5 \times 10^{-9}$  en los parámetros (F3). Con  $\gamma = 0.08$ , una QNN estándar entrenada sin ruido colapsa bajo T1 (0.771); entrenada bajo T1 se recupera (0.944).

Las predicciones que fallaron:

- la codificación arccos no mejoró la precisión;
- el gemelo cuadrático clásico no quedó a menos de 1 punto de la QNN con la regularización por defecto;
- el T1 desigual le costó al filtro 1.5 puntos en una corrida (límite: 1 punto);
- el peso 2 fue menos preciso que el peso 1 (por 3.7 puntos);
- entrenado bajo T1, el modelo de peso 2 sin filtro quedó 0.6 puntos por delante del filtrado;
- en los estudios de la Sec. 4: la lectura  $\langle Z_i Z_j \rangle$  no ayudó al peso 2 (dos predicciones), el filtro no fue neutral con entrenamiento con ruido y esa lectura, la ganancia del filtro en una dispersión de T1 tuvo un IC que tocó el cero, el peso 2 no fue más sensible a la dispersión que el peso 1, y la codificación por productos de pares tuvo un techo de información menor que la unaria.

**Réplica en 60 corridas.** Las comparaciones clave se repitieron con 5 semillas  $\times$  3 particiones  $\times$  4 conjuntos. La Tabla 1 y la Fig. 1 dan los resultados; las cinco predicciones pre-registradas de esta réplica se cumplieron.

| lectura                          | E (peso 1) |          | W (peso 2) |          |
|----------------------------------|------------|----------|------------|----------|
|                                  | con qang   | sin qang | con qang   | sin qang |
| exacta (sin ruido)               | 0.952      |          | 0.922      |          |
| T1, entrenada sin ruido          | 0.952      | 0.928    | 0.922      | 0.757    |
| T1, entrenada con el ruido       | 0.952      | 0.951    | 0.922      | 0.924    |
| T1 desigual, entrenada sin ruido | 0.947      | 0.921    | 0.916      | 0.732    |
| desfase, entrenada sin ruido     | 0.918      | 0.792    | 0.882      | 0.656    |
| desfase, entrenada con el ruido  | 0.950      |          | 0.926      |          |

Tabla 1: Precisión de prueba media en 60 corridas por modelo, con y sin el filtro qg.

Las diferencias pareadas (media, IC 95 % entre semillas) fueron las siguientes.

- *Ganancia del filtro, entrenada sin ruido, bajo T1:* +2.5 [0.8, 4.1] puntos (E) y +16.5 [11.8, 21.2] (W). El modelo filtrado quedó adelante o empatado en 50/60 (E) y 56/60 (W) corridas.
- *Con entrenamiento con ruido:* +0.1 [−0.6, 0.9] (E) y −0.1 [−0.8, 0.5] (W).
- *Pérdida por T1 desigual:* 0.5 [−0.2, 1.2] (E) y 0.7 [0.0, 1.3] (W). Los 1.5 puntos de la corrida anterior estaban en el extremo alto de la dispersión.
- *Desfase, entrenamiento con ruido menos sin ruido, con filtro:* +3.2 (E) y +4.5 (W), ambos IC por encima de 0.

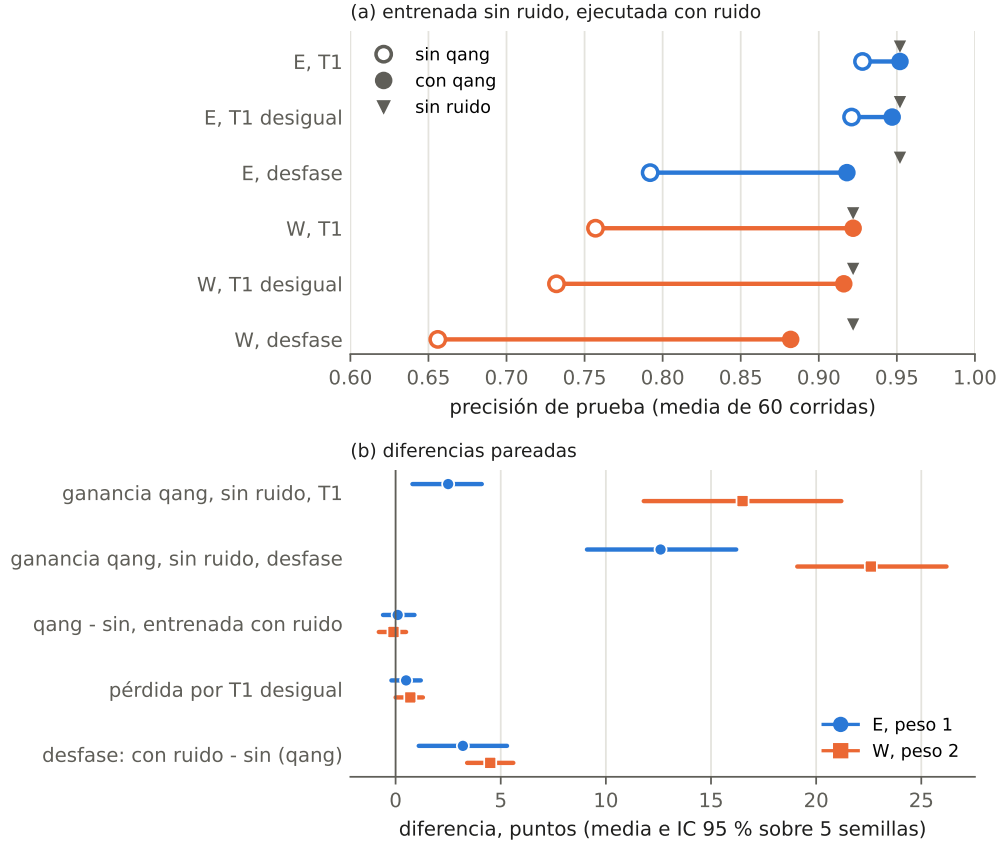


Figura 1: (a) Precisión de redes entrenadas sin ruido y ejecutadas con ruido, sin (vacío) y con (lleno) el filtro; los triángulos marcan la precisión sin ruido. (b) Diferencias pareadas, media e IC 95 % sobre 5 semillas.

- *Con desfase, un modelo entrenado sin ruido igual gana +12.6 (E) y +22.6 (W) puntos con el filtro, que elimina la parte T1 del ruido combinado.*
- *Peso 2 frente a peso 1:  $-3.0 [-4.0, -2.0]$  puntos.*

### Lectura, codificación y dispersión de T1 (60 corridas cada una).

- *Lectura de correlaciones.* Agregar las  $\binom{5}{2} = 10$  correlaciones  $\langle Z_i Z_j \rangle$  hace que la lectura de peso 2 sea una función lineal de toda la distribución del sector (rango 10 en lugar de 5). No ayudó:  $-0.6 [-1.5, 0.4]$  puntos. Entrenado bajo T1 con 15 pesos de lectura, el modelo *sin* filtro fue 1.2  $[0.4, 2.0]$  puntos mejor. Los disparos decaídos todavía informan dónde estaban las excitaciones, y el filtro descarta esa información. El filtro sigue siendo exacto para esta lectura, y entrenando sin ruido suma +14.9 puntos.
- *Codificación.* La mejor forma cuadrática sin restricciones en las amplitudes de productos de pares llega a 0.928, 2.4  $[1.7, 3.0]$  puntos por debajo del mismo techo para las amplitudes unarias (0.952). La normalización de los productos distorsiona los datos. Cargar los datos de peso 1 en cinco estados de peso 2 (un “anillo” de pares  $(k, k+1)$ ) da 0.943, a 1.0 punto del peso 1 y 2.3  $[0.7, 3.9]$  puntos por encima de los pares. Con esta codificación la ganancia del filtro bajo T1 es la mayor vista, +23.9 puntos.
- *Dispersión de T1.* Con  $\gamma_q = 0.08(1 + s u_q)$  y  $u_q$  repartido en  $[-1, 1]$ , la pérdida del filtro fue de 0.3/0.2 puntos (peso 1/2) con  $s = 0.5$  y 1.4/1.3 con  $s = 1$ . El filtro solo mantiene

la pérdida por debajo de un punto hasta  $s \approx 0.8$ , es decir, una dispersión de  $\pm 80\%$  de las tasas  $1/T_1$  alrededor de su media.

**Modelos de ruido de equipos.** El modelo de peso 1 se compiló a Qiskit: una cascada de 4 compuertas RBS carga las amplitudes unarias con signo, y siguen las 15 RBS entrenadas; el circuito coincide con el modelo a  $10^{-16}$ . Se corrió con 30 entradas de prueba de iris y 1000 disparos cada una, leídas con y sin filtro de los mismos disparos. Los modelos de ruido fueron los de tres equipos de IBM (backends simulados con su calibración, en Aer) y el simulador en la nube de IonQ con los modelos de ruido Aria-1 y Forte-1. En los cinco la precisión fue 0.933 con y sin filtro, igual a la precisión sin ruido: los márgenes de este modelo son amplios. El error del *qgz* medido bajó de 0.138–0.206 sin filtro a 0.040–0.072 con filtro (entre 2.9 y 3.6 veces), conservando el 69–74 % de los disparos. Son modelos de ruido, no equipos; las corridas en hardware están preparadas, con confirmación de costo, y pendientes.

## 5 Discusión

La diferencia que hace el filtro depende de cómo se entrenó la red. Entrenada en simulador y ejecutada con relajación, el filtro es decisivo. Su efecto crece con el peso, porque un estado de peso  $k$  decae  $k$  veces más rápido. Entrenada con un modelo de ruido calibrado, una red sin filtro aprende el decaimiento, y el filtro no agrega nada medible. Lo que aporta el filtro es, por lo tanto, de procedimiento y no una ganancia de precisión sobre la mejor alternativa. Con T1 igual hace exactamente válido el entrenamiento sin ruido, sin modelo de ruido ni corridas de entrenamiento con ruido, y su costo (los disparos descartados) se conoce antes de ejecutar. El T1 desigual y el desfase no se corrigen. En hardware con mucha dispersión de T1 o desfase fuerte, el filtro complementa un modelo de ruido y no lo reemplaza. De los estudios anteriores salen dos reglas prácticas. Hasta una dispersión de  $\pm 80\%$  de  $1/T_1$  entre qubits, una red entrenada en simulador puede ejecutarse solo con el filtro, perdiendo menos de un punto. Si en cambio la red se entrena con el ruido calibrado, y sobre todo si su lectura tiene muchos parámetros, los disparos decaídos son datos útiles y el filtro puede costar alrededor de un punto. Para modelos de peso 2, importa más cómo se cargan los datos que qué se lee.

**Limitaciones.** Todo el ruido es simulado, incluidos los modelos de ruido de equipos. Hay una arquitectura por peso, cinco qubits y conjuntos pequeños en los que una muestra de prueba vale 2–3 puntos. La codificación en anillo usa solo cinco de los diez estados de peso 2, así que un mejor uso del sector más grande sigue abierto. No se afirma ventaja cuántica: estas redes son simulables clásicamente. Una corrida en hardware con el filtro es la siguiente prueba natural.

## Disponibilidad de código y datos

Modelos: `qang.qml` en `qang` 0.6.0 (`pip install qang`), solo NumPy, con `WeightQNN` (pesos 1 y 2, lecturas "z" y "zz", codificaciones "pairs" y "ring"), `kept_fraction` y `compare_qang`. Script listo para hardware: `examples/qnn_hardware_qg.py` (IBM e IonQ). Estudios: `examples/qnn*_qg.py` y RESEARCH\_NOTES §75–§82, §85 y §87, con pruebas en `tests/`. Los cuadernos Colab `notebooks/qang_qml-es.ipynb` (español) y `qang_qml.ipynb` (inglés) instalan la librería desde PyPI. Repositorio: <https://github.com/Viny2030/qang>.

```
from qang.qml import WeightQNN
m = WeightQNN(n_qubits=5, weight=2).fit(X_train, y_train)
m.score(X_test, y_test, gamma=0.08, qang=True) # = precision sin ruido
m.score(X_test, y_test, gamma=0.08, qang=False) # lectura cruda
```

## Referencias

- [1] Vicente Humberto Monteverde. The Qang (qg): A unified angular-probability unit and metric for parametric quantum circuit design. 2026. doi: 10.5281/zenodo.22832150.
- [2] Vicente Humberto Monteverde. The register-mean polar bias as a symmetry witness: error mitigation, constrained optimization and qubit characterization in qg units. *Zenodo preprint*, 2026. doi: 10.5281/zenodo.23036963.
- [3] Hanrui Wang, Jiaqi Gu, Yongshan Ding, Zirui Li, Frederic T. Chong, David Z. Pan, and Song Han. QuantumNAT: quantum noise-aware training with noise injection, quantization and normalization. In *Proceedings of the 59th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2022. doi: 10.1145/3489517.3530400.
- [4] Xavier Bonet-Monroig, Ramiro Sagastizabal, M. Singh, and Thomas E. O’Brien. Low-cost error mitigation by symmetry verification. *Physical Review A*, 98:062339, 2018.
- [5] Sam McArdle, Xiao Yuan, and Simon Benjamin. Error-mitigated digital quantum simulation. *Physical Review Letters*, 122:180501, 2019.
- [6] Jonas Landman, Natansh Mathur, Yun Yvonna Li, Martin Strahm, Skander Kazdaghli, Anupam Prakash, and Iordanis Kerenidis. Quantum methods for neural networks and application to medical image classification. *Quantum*, 6:881, 2022. doi: 10.22331/q-2022-12-22-881.
- [7] M. Cerezo et al. Does provable absence of barren plateaus imply classical simulability? Or, why we need to rethink variational quantum computing, 2023. arXiv:2312.09121.